

Git & GitHub

Инструкция по работе с системой контроля версий

Техническая документация v1.0

Development Team

7 июля 2026 г.

Для одиночной и командной разработки

Оглавление

1	Подготовка на GitHub	3
1.1	Создание репозитория	3
1.1.1	Пошаговая инструкция	3
2	Настройка локального проекта	4
2.1	Файл .gitignore	4
2.1.1	Создание файла	4
2.1.2	Правила игнорирования	4
3	Команды в терминале	5
3.1	Инициализация Git	5
3.2	Работа с ветками	5
3.2.1	Основные команды	5
3.3	Связь с GitHub	5
3.3.1	Подключение удаленного репозитория	5
3.3.2	Проверка подключения	5
3.4	Добавление файлов	5
3.4.1	Добавление конкретного файла	5
3.4.2	Добавление всех файлов	6
3.5	Создание коммитов	6
3.5.1	Базовый коммит	6
3.5.2	Коммит всех изменённых файлов	6
3.5.3	Добавить файлы к последнему коммиту	6
3.5.4	Изменить сообщение последнего коммита	6
3.5.5	Коммит с указанием автора	6
3.5.6	Пустой коммит (для тестов)	6
3.5.7	Пропустить pre-commit хуки	6
3.6	Отправка на GitHub	7
3.6.1	Первая отправка	7
3.6.2	Последующие отправки	7
3.7	Получение изменений с GitHub	7
3.7.1	Получение и слияние	7
3.7.2	Только скачать (без слияния)	7
3.8	Просмотр информации	7
3.8.1	Статус файлов	7
3.8.2	Все ветки	7
3.8.3	История коммитов	7
3.8.4	Краткая история	7

3.9	Отмена изменений	8
3.9.1	Отменить изменения в файле	8
3.9.2	Убрать файл из индекса	8
3.9.3	Отменить последний коммит (сохранить файлы)	8
3.9.4	Полностью отменить коммит	8
3.10	Удаление файлов	8
3.10.1	Удалить из Git (оставить на диске)	8
3.10.2	Удалить из Git и с диска	8
3.11	Удаление веток	8
3.11.1	Удалить локальную ветку	8
3.11.2	Принудительно удалить	8
4	Последующие коммиты (ежедневно)	9
4.1	Ежедневный workflow	9
4.1.1	Стандартная последовательность	9
5	Работа в команде	10
5.1	Перед началом работы	10
5.2	Создание ветки для задачи	10
5.3	Работа, коммиты, отправка	10
5.4	Создание Pull Request на GitHub	10
5.4.1	Пошаговая инструкция	10
5.5	Ожидание ревью	11
5.5.1	Процесс ревью	11
5.6	После одобрения	11
5.6.1	Слияние и очистка	11
6	Заключение	12

1. Подготовка на GitHub

1.1 Создание репозитория

1.1.1 Пошаговая инструкция

1. Зайти на **github.com**
2. Нажать “+” → “New repository”
3. Заполнить форму:
 - **Repository name:** telegram-body-changer-bot
 - **Description:** (необязательно)
 - **Public/Private:** на ваш выбор
 - **НЕ ставить галочки** (README, .gitignore, license)
4. Нажать “Create repository”
5. Скопировать ссылку репозитория

2. Настройка локального проекта

2.1 Файл .gitignore

2.1.1 Создание файла

Создать файл `.gitignore` в корне проекта.

2.1.2 Правила игнорирования

Добавить следующие правила:

```
1 .idea/  
2 .vscode/  
3 venv/  
4 __pycache__/  
5 *.py[cod]
```

Листинг 2.1: Содержимое `.gitignore`

3. Команды в терминале

3.1 Инициализация Git

```
1 git init
```

3.2 Работа с ветками

3.2.1 Основные команды

- Переименовать текущую ветку: `git branch -M main`
- Создать новую ветку: `git branch feature-name`
- Переключиться на ветку: `git checkout feature-name`
- Создать ветку и сразу переключиться: `git checkout -b feature-name`

3.3 Связь с GitHub

3.3.1 Подключение удаленного репозитория

```
1 git remote add origin https://github.com/      /  
      .git
```

3.3.2 Проверка подключения

```
1 git remote -v
```

3.4 Добавление файлов

3.4.1 Добавление конкретного файла

```
1 git add filename.txt
```

3.4.2 Добавление всех файлов

```
1 git add .
```

Точка в конце обязательна!

3.5 Создание коммитов

3.5.1 Базовый коммит

```
1 git commit -m " "
```

Сообщение в кавычках!

3.5.2 Коммит всех изменённых файлов

```
1 git commit -am " "
```

3.5.3 Добавить файлы к последнему коммиту

```
1 git commit --amend
```

3.5.4 Изменить сообщение последнего коммита

```
1 git commit --amend -m " "
```

3.5.5 Коммит с указанием автора

```
1 git commit -m " " --author=" <email@example.com> "
```

3.5.6 Пустой коммит (для тестов)

```
1 git commit --allow-empty -m " "
```

3.5.7 Пропустить pre-commit хуки

```
1 git commit -m " " --no-verify
```

3.6 Отправка на GitHub

3.6.1 Первая отправка

```
1 git push -u origin main
```

3.6.2 Последующие отправки

```
1 git push
```

3.7 Получение изменений с GitHub

3.7.1 Получение и слияние

```
1 git pull origin main
```

3.7.2 Только скачать (без слияния)

```
1 git fetch origin
```

3.8 Просмотр информации

3.8.1 Статус файлов

```
1 git status
```

3.8.2 Все ветки

```
1 git branch -a
```

3.8.3 История коммитов

```
1 git log
```

3.8.4 Краткая история

```
1 git log --oneline
```


3.9 Отмена изменений

3.9.1 Отменить изменения в файле

```
1 git checkout -- filename.txt
```

3.9.2 Убрать файл из индекса

```
1 git reset HEAD filename.txt
```

3.9.3 Отменить последний коммит (сохранить файлы)

```
1 git reset --soft HEAD~1
```

3.9.4 Полностью отменить коммит

```
1 git reset --hard HEAD~1
```

Все изменения пропадут!

3.10 Удаление файлов

3.10.1 Удалить из Git (оставить на диске)

```
1 git rm --cached filename.txt
```

3.10.2 Удалить из Git и с диска

```
1 git rm filename.txt
```

3.11 Удаление веток

3.11.1 Удалить локальную ветку

```
1 git branch -d feature-name
```

3.11.2 Принудительно удалить

```
1 git branch -D feature-name
```

4. Последующие коммиты (ежедневно)

4.1 Ежедневный workflow

4.1.1 Стандартная последовательность

1. `git add .`
2. `git commit -m "Что сделал"`
3. `git push`

5. Работа в команде

5.1 Перед началом работы

```
1 git pull origin main
```

Получить последние изменения от команды.

5.2 Создание ветки для задачи

```
1 git checkout -b task-123
```

5.3 Работа, коммиты, отправка

1. `git add .`
2. `git commit -m "Task 123: добавлена функция"`
3. `git push -u origin task-123`

5.4 Создание Pull Request на GitHub

5.4.1 Пошаговая инструкция

1. Зайти на страницу репозитория на GitHub
2. Нажать вкладку “Pull requests”
3. Нажать кнопку “New pull request”
4. Выбрать ветки:
 - **base:** main (куда сливаем изменения)
 - **compare:** task-123 (откуда берём изменения)
5. Нажать “Create pull request”
6. Заполнить описание:
 - **Title:** Краткое название задачи (например, “Task 123: Добавлена функция X”)

- **Description:** Подробное описание изменений:

- Что было сделано
- Зачем это нужно
- Как протестировать
- Скриншоты (если есть UI изменения)

7. Назначить ревьюеров (справа в разделе “Reviewers”)

8. Добавить labels (если нужно)

9. Нажать “Create pull request”

5.5 Ожидание ревью

5.5.1 Процесс ревью

- Ревьюеры оставляют комментарии
- Внести правки локально:
 - `git checkout task-123` (переключиться на ветку задачи)
 - Внести изменения
 - `git add .`
 - `git commit -m "Task 123: исправлены замечания"`
 - `git push` (автоматически обновит PR)
- Ответить на комментарии в GitHub

5.6 После одобрения

5.6.1 Слияние и очистка

- Нажать “Merge pull request” на GitHub
- Удалить ветку (опционально): `git branch -d task-123`

6. Заключение

Система контроля версий Git является неотъемлемым инструментом современного разработчика. Данная инструкция покрывает основные сценарии работы как для одиночной разработки, так и для командной работы.

Ключевые преимущества использования Git:

1. **История изменений** — полная отслеживаемость всех модификаций кода
2. **Ветвление** — возможность параллельной разработки функций
3. **Командная работа** — эффективная совместная работа через Pull Request
4. **Безопасность** — возможность отката к любой предыдущей версии
5. **Резервное копирование** — код хранится на удаленном сервере

Git & GitHub — Система контроля версий для профессионалов.